

Atividade Prática — Arte Generativa 3D

Escultura Generativa

1. Objetivo

Nesta atividade você vai construir uma cena 3D no navegador que gera, por código, uma escultura abstrata: um conjunto de pontos no espaço, conectados entre si, girando continuamente, com uma paleta de cores diferente a cada geração. Você vai usar um assistente de IA generativa (ChatGPT, Codex ou similar) como apoio durante todo o desenvolvimento.

Diferente das atividades anteriores, aqui não existe uma "resposta certa" a ser alcançada — o resultado visual depende do algoritmo que você escrever. O objetivo é treinar a estruturação de uma solução com uma biblioteca gráfica 3D e, ao mesmo tempo, pensar em lógica de programação de um jeito criativo: código que gera arte.

2. Sobre a proposta

Arte generativa é qualquer arte criada com o apoio de um sistema — no nosso caso, um algoritmo — que introduz alguma regra ou aleatoriedade na composição, em vez de um resultado desenhado ou modelado à mão. Não existe uma imagem de referência fixa para esta atividade: a escultura final depende inteiramente das regras que você programar.

💡 O que importa aqui é a regra, não o resultado exato

Não existe uma imagem de referência para copiar nesta atividade — o visual final da sua escultura é consequência do seu código.

O que será avaliado é a lógica por trás da geração (veja a seção 9), não se a escultura ficou "bonita" no sentido tradicional.

Gerar pontos totalmente aleatórios sem nenhuma regra é o requisito mínimo — pense em padrões (espirais, grades, esferas, camadas) para deixar o resultado mais interessante.

3. A tecnologia: Three.js

Three.js é a biblioteca mais usada para gráficos 3D no navegador. Ela já vai estar disponível localmente, na pasta vendor/ do projeto (sem depender de internet), incluindo o módulo opcional OrbitControls, caso você queira implementá-lo como melhoria.

Referência rápida das peças que você vai mais usar:

Recurso do Three.js	Para que serve
<code>THREE.Scene()</code>	O "palco" onde todos os objetos 3D existem.
<code>THREE.PerspectiveCamera(...)</code>	Define o ponto de vista de onde a cena é vista.
<code>THREE.WebGLRenderer(...)</code>	Desenha a cena na tela, dentro de um <canvas>.
<code>THREE.SphereGeometry</code> / <code>BoxGeometry</code> / <code>TubeGeometry</code>	Formas prontas para representar os pontos ou as conexões entre eles.
<code>THREE.MeshStandardMaterial({ color, ... })</code>	Define a aparência de um objeto (cor, brilho, transparência).
<code>THREE.AmbientLight</code> / <code>THREE.PointLight</code>	Luzes — sem alguma luz na cena, os materiais "Standard" aparecem pretos.
<code>requestAnimationFrame(loop)</code>	Roda a função de animação a cada quadro, usada para girar a escultura continuamente.



Como rodar seu projeto

Three.js é importado como módulo ES (`import ... from './vendor/three.module.js'`), e o navegador bloqueia esse tipo de importação quando o arquivo é aberto direto pelo protocolo `file://`.

No VS Code, clique com o botão direito no `index.html` e escolha "Open with Live Server". Sem isso, a cena não vai carregar.

Referencie sempre os arquivos locais da pasta `vendor/` — não use links de CDN, que são bloqueados pela rede da escola.

4. Requisitos obrigatórios

Sua escultura generativa precisa ter, no mínimo:

Requisito	Descrição
Cena 3D funcional	Câmera, iluminação e renderer configurados, ocupando a tela toda.
Geração algorítmica dos pontos	Um conjunto de pontos no espaço 3D, com posições calculadas por código (fórmula, laço com aleatoriedade, ou ambos) — nunca digitados um por um à mão.
Conexão entre os pontos	Os pontos são ligados por linhas, tubos ou outra forma geométrica, formando o "esqueleto" da escultura.
Rotação contínua	A escultura gira lentamente e sem parar, usando o loop de animação do Three.js.
Paleta de cores gerada por código	As cores dos elementos são geradas (não fixas no código), mudando a cada nova escultura.
Botão "Gerar nova escultura"	Ao clicar, toda a lógica roda de novo com novos valores aleatórios, substituindo a escultura anterior.

5. Melhorias — escolha as suas

Além dos requisitos da seção 4, você deve implementar pelo menos 3 melhorias que julgar interessantes. Não existe uma lista fechada e obrigatória — pense em como deixar a experiência mais rica ou mais interativa. Algumas ideias para começar:

- Sliders para controlar parâmetros da geração: quantidade de pontos, velocidade de rotação, espessura das conexões.
- Controle de câmera orbital (OrbitControls do próprio Three.js), para o usuário girar e dar zoom na escultura com o mouse.
- Botão para exportar a escultura atual como imagem PNG.
- Sistema de "semente" (seed): guardar um código que permite gerar a mesma escultura de novo mais tarde.
- Fundo customizado: gradiente, céu estrelado, ou neblina (fog) do Three.js.
- Mais de um "estilo" de geração para o usuário escolher (ex.: esfera de pontos, espiral, grade distorcida).
- Efeito de partículas flutuando ao redor da escultura.
- Luz que muda de cor com o tempo, iluminando a escultura de formas diferentes.

6. Entrega

- O(s) arquivo(s) do projeto (HTML, CSS, JS) e a pasta vendor/.
- Print de tela (ou vídeo curto) mostrando pelo menos três esculturas diferentes geradas pelo botão "Gerar nova escultura", e as melhorias implementadas.

Lembrete importante

Pontos e conexões precisam ser gerados por código — copiar uma escultura pronta de exemplo, com coordenadas fixas no código, não atende ao requisito da seção 4.

Sempre teste com o Live Server rodando — abrir o arquivo direto no navegador vai impedir o carregamento do Three.js.

Clique em "Gerar nova escultura" várias vezes durante o teste — é assim que se percebe se a paleta de cores e os pontos realmente estão variando.